



UANL

UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN



FIME

FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

Universidad Autónoma de Nuevo León

Facultad de Ingeniería Mecánica y Eléctrica

Lenguajes de Programación y Lab

Tarea 1

Semestre: 3ro

Hora: M4

Grupo: 025

Salón: 3109

Periodo: Enero – Junio

Docente: Ing. Serene Guadalupe Pinal Torres

Jordan Emanuel Ruedas Vazquez 2103677 ITS

Héctor Miguel Enríquez Nungaray. 2042781 IAS

Betsy Aracely Heredia López. 2054413. IAS

Carlos Yahir Hernández Silva 1975208 ITS

Martes 04 de Febrero del 2025

Monterrey, Nuevo León

Línea del Tiempo de los Lenguajes de Programación (1995-2025)

Este documento presenta una línea del tiempo con los lenguajes de programación más influyentes desde 1995 hasta 2025. Se destacan sus características clave, logros y aplicaciones más relevantes.

1995: JavaScript

Inicialmente creado por Netscape, JavaScript se convirtió en el estándar para interactividad en páginas web. Hoy en día, es la base de frameworks modernos como React, Angular y Vue, esenciales para el desarrollo web.



1995: Java

Desarrollado por Sun Microsystems, Java es uno de los lenguajes más influyentes en la industria. Su lema 'escribe una vez, ejecuta en cualquier parte' permitió que fuera adoptado en desarrollo empresarial, dispositivos móviles y grandes sistemas de backend.

1995: PHP

Lenguaje de scripting diseñado para la creación de sitios web dinámicos. PHP ha sido fundamental en el desarrollo de plataformas como WordPress y Laravel.

2000: C#

Microsoft introdujo C# como parte de su plataforma .NET, permitiendo el desarrollo de aplicaciones empresariales, juegos en Unity y sistemas multiplataforma.



2001: Visual Basic .NET

Evolución del clásico Visual Basic, modernizando su sintaxis y adaptándose al .NET Framework.

2003: Scala

Un lenguaje que combina programación funcional y orientada a objetos, usado en el desarrollo backend con frameworks como Akka y Play.



2009: Go

Creado por Google, Go es altamente eficiente y se ha convertido en el estándar para infraestructura en la nube y sistemas de alto rendimiento.

2009: Rust

Con enfoque en seguridad y rendimiento, Rust se ha vuelto popular en desarrollo de sistemas y videojuegos, superando a C++ en ciertos aspectos.



2010: Dart

Google introdujo Dart para desarrollo web y móvil, destacando especialmente con el framework Flutter.

2011: Kotlin

Lenguaje oficial para Android desde 2017, reemplazando gradualmente a Java en el ecosistema de desarrollo móvil.



2012: Julia

Potente para computación científica y análisis

de datos, con rendimiento similar a C.

2014: Swift

Apple desarrolló Swift para hacer el desarrollo en iOS y macOS más accesible y seguro.



TS

2016: Elixir

Lenguaje funcional basado en Erlang, utilizado en aplicaciones de alta concurrencia como WhatsApp y Discord.

2018: Raku

Evolución de Perl con mayor expresividad y mejoras de rendimiento.

2020: F#

Lenguaje funcional de Microsoft, ampliamente usado en modelado financiero y análisis de datos.



2023: Mojo

Optimizado para inteligencia artificial y aprendizaje automático, combinando Python con rendimiento de C.



2015: TypeScript

Extiende JavaScript con tipado estático, facilitando la escalabilidad en grandes proyectos de software.

2017: Crystal

Similar a Ruby, pero con rendimiento comparable a C, ideal para aplicaciones web.

2019: Ballerina

Diseñado para integrar servicios y APIs de forma sencilla.



2021: Zig

Alternativa moderna a C, con mejor manejo de memoria y seguridad.

En la última década, los lenguajes de programación han experimentado avances notables que responden a las crecientes demandas de eficiencia, seguridad y escalabilidad en el desarrollo de software. Innovaciones recientes han dado lugar a lenguajes y tecnologías emergentes, como Mojo, Zig y F#, diseñados para abordar necesidades específicas en áreas como inteligencia artificial, gestión de memoria y modelado financiero. Estas nuevas herramientas han surgido en un entorno marcado por la evolución constante de la tecnología, en el que la interoperabilidad, la facilidad de mantenimiento y la integración con infraestructuras existentes juegan un papel fundamental en la adopción y éxito de los lenguajes.

Al mismo tiempo, los lenguajes tradicionales como JavaScript, Java, C# y PHP continúan siendo ampliamente utilizados, ya que han demostrado su robustez y versatilidad en aplicaciones empresariales, desarrollo web y sistemas integrados. La incorporación de nuevos paradigmas, como la programación funcional y el tipado estático en lenguajes originalmente dinámicos (por ejemplo, TypeScript extendiendo JavaScript), ha contribuido a reducir errores y mejorar la escalabilidad de proyectos de gran envergadura. Este contexto nos permite analizar las ventajas y desventajas que se han observado con el paso de los años, destacando cómo cada lenguaje se adapta a los retos modernos del desarrollo de software.

El análisis de estas tendencias muestra que, mientras las innovaciones tecnológicas impulsan una mayor eficiencia en el procesamiento y la gestión de datos, también se generan desafíos en términos de compatibilidad y curva de aprendizaje. Las empresas deben sopesar los beneficios de adoptar lenguajes de nueva generación frente a la consolidación de tecnologías que ya forman parte de su infraestructura. Esta dualidad es crucial para comprender el impacto que tienen las decisiones tecnológicas en la competitividad y sostenibilidad de las organizaciones a nivel global.

➤ **Ventajas**

Una de las principales ventajas de los lenguajes emergentes es la **eficiencia en el rendimiento**. Por ejemplo, Mojo ha sido diseñado para tareas de inteligencia artificial, ofreciendo la flexibilidad de Python combinada con el rendimiento de C, lo que resulta especialmente útil en la creación de modelos de aprendizaje automático que requieren procesar grandes volúmenes de datos en tiempo real. Además, lenguajes como Zig se destacan por mejorar la seguridad en la gestión de memoria, reduciendo errores críticos como los desbordamientos de búfer, lo que se traduce en aplicaciones más seguras y robustas en ámbitos como la ciberseguridad y el desarrollo de sistemas operativos.

Otra ventaja importante es el **enfoque en la escalabilidad y seguridad**. Lenguajes como F# permiten realizar análisis de datos y modelado financiero con un alto grado de precisión gracias a su tipado fuerte y funcionalidad avanzada, facilitando la implementación de software que pueda crecer y adaptarse a las exigencias del mercado. Asimismo, la integración de herramientas de tipado estático en lenguajes tradicionales—como lo hace TypeScript con JavaScript—reduce la cantidad de errores en el desarrollo, haciendo que el código sea más predecible y fácil de mantener en proyectos de gran escala.

Adicionalmente, la **orientación hacia la optimización de procesos** se evidencia en el surgimiento de nuevos paradigmas y metodologías que permiten a los desarrolladores escribir código más limpio y eficiente. Estas innovaciones facilitan la integración de técnicas modernas de control y análisis de datos, lo que se traduce en mejoras significativas en el tiempo de desarrollo y en la capacidad de respuesta de las aplicaciones frente a las demandas del entorno digital actual.

➤ **Desventajas**

Sin embargo, a pesar de estas ventajas, la adopción de nuevos lenguajes y tecnologías conlleva importantes ****desafíos de compatibilidad****. Muchas organizaciones cuentan con infraestructuras basadas en lenguajes tradicionales y, en consecuencia, la migración a lenguajes emergentes puede resultar costosa y compleja. Esto genera una barrera significativa, ya que la integración de nuevas tecnologías requiere no solo de una actualización del software existente, sino también de una inversión considerable en capacitación y adaptación de los equipos de desarrollo.

Otra desventaja es la ****curva de aprendizaje pronunciada**** asociada a muchos de los lenguajes modernos. Aunque lenguajes como Mojo, Zig y F# ofrecen mejoras en rendimiento y seguridad, su adopción puede verse limitada por la falta de experiencia generalizada en estos entornos. Los desarrolladores deben invertir tiempo y recursos en formarse y en familiarizarse con nuevos paradigmas de programación, lo que puede retrasar la implementación de proyectos en entornos donde la rapidez es crucial.

Además, la ****fragmentación del ecosistema de desarrollo**** constituye un reto importante. Con la proliferación de lenguajes y frameworks, la interoperabilidad entre diferentes tecnologías se vuelve cada vez más complicada. Esto puede dificultar la colaboración entre equipos y la integración de soluciones, especialmente en proyectos de gran envergadura donde se requieren múltiples tecnologías que deben interactuar de manera eficiente. La dispersión de conocimientos y la falta de un estándar unificado pueden generar problemas de compatibilidad y aumentar la complejidad en el mantenimiento a largo plazo.

Conclusión

En resumen, el período comprendido entre 2020 y 2025 ha sido testigo de una evolución dinámica en el campo de los lenguajes de programación, caracterizada por la aparición de herramientas que ofrecen mejoras en rendimiento, seguridad y escalabilidad. Mientras que los lenguajes emergentes como Mojo, Zig y F# han traído consigo innovaciones técnicas que responden a las necesidades de la inteligencia artificial, la ciberseguridad y el análisis financiero, también han planteado desafíos en cuanto a compatibilidad, curva de aprendizaje y fragmentación del ecosistema tecnológico. Este análisis evidencia que la elección de un lenguaje de programación no solo depende de sus características técnicas, sino también del contexto y la infraestructura existente en cada organización.

A medida que la tecnología continúa avanzando, es esencial que tanto las empresas como los profesionales del desarrollo de software se mantengan actualizados en las últimas tendencias y herramientas. La capacidad para integrar tecnologías nuevas con infraestructuras ya consolidadas será crucial para asegurar la competitividad y la sostenibilidad en el mercado global. En este sentido, la formación continua y la inversión en capacitación se presentan como elementos indispensables para aprovechar al máximo las ventajas que ofrecen los lenguajes emergentes y mitigar sus desventajas.

Por último, la reflexión sobre el impacto de estos lenguajes en la industria tecnológica nos invita a considerar la importancia de una estrategia de innovación equilibrada, en la que se combinen tecnologías probadas con nuevas herramientas de vanguardia. Esta integración permitirá no solo optimizar el desarrollo de software, sino también crear un entorno de trabajo en el que la calidad y la eficiencia sean los pilares fundamentales para el éxito a largo plazo. Los desafíos que surgen en este contexto deben ser abordados con una visión estratégica que contemple tanto los beneficios inmediatos como la adaptación a las futuras demandas del mercado.

Referencias Bibliográficas

Ahire, S., Golhar, D., & Waller, M. A. (1996). *The impact of quality management practices on performance and competitive advantage*. *International Journal of Quality & Reliability Management, 13*(7), 651-664. <https://doi.org/10.1108/02656719610139266>

Kumar, S., De Grosbois, D., Choisine, F., & Kumar, V. (2008). *Towards a new measure of quality management*. *International Journal of Production Economics, 110*(2), 240-252. <https://doi.org/10.1016/j.ijpe.2007.10.008>

Widodo, A. (2011). *Critical success factors for implementing total quality management in the Indonesian government sector*. *Total Quality Management & Business Excellence, 22*(9-10), 1057-1073. <https://doi.org/10.1080/14783363.2010.520335>